Advanced Object Oriented Programming

Xingjie ZHANG

아주대학교
AJOU UNIVERSITY

高级面向对象编程

张行健

아주대학교
AJOU UNIVERSITY

아주대학교
AJOU UNIVERSITY

Operators

Instructor: Xingjie ZHANG

Email : xzhang@ajou.ac.kr

Wechat : runningSnaile

아주대학교
AJOU UNIVERSITY

操作符

讲师：张行杰 Email : xzhang@ajou.ac.kr 微信：runningSnaile

CONTENTS

01 Operators

02 Arithmetic Operators

03 Assignment Operators

04 Increment and Decrement Operators

05 Comparison, Logical, Conditional

아주대학교 AJOU UNIVERSITY

目录

符 02 算术运算符

运算符 04 自增和自减运算符 05 比较运
逻辑运算符、条件运算符

아주대학교 AJOU UNIVERSITY

01

Operators

01

操作符

아주대학교 AJOU UNIVERSITY

What are Operators?

- **Operators** are fundamental symbols in programming languages like Java that are used to perform various operations on variables and values.
- These operations can range from simple mathematical calculations to complex logical judgments and bit-level manipulations.

Arithmetic
Used for mathematical calculations (addition, subtraction, multiplication, division, remainder)

Assignment
Used to assign values to variables, including shorthand compound assignments

Increment/Decrement
Used to increase or decrease the value of a variable by one

Comparison
Used to compare two values and return a boolean result (true or false)

Logical
Used to combine or negate boolean expressions, commonly found in conditional statements

Conditional
A shorthand for simple if-else statements, allowing a value to be chosen based on a condition

Bitwise
Used to perform operations directly on the individual bits of integer types

Shift
Used to shift the bits of a number to the left or right

instanceof
Used for type checking, to determine if an object is an instance of a particular class or interface

什么是运算符?

- 运算符是像 Java 这样的编程语言中的基本符号，用于对变量和值执行各种操作。
- 这些操作可以从简单的数学计算到复杂的逻辑判断和位级操作不等。

Arithmetic
Used for mathematical calculations (addition, subtraction, multiplication, division, remainder)

Assignment
Used to assign values to variables, including shorthand compound assignments

Increment/Decrement
Used to increase or decrease the value of a variable by one

Comparison
Used to compare two values and return a boolean result (true or false)

Logical
Used to combine or negate boolean expressions, commonly found in conditional statements

Conditional
A shorthand for simple if-else statements, allowing a value to be chosen based on a condition

Bitwise
Used to perform operations directly on the individual bits of integer types

Shift
Used to shift the bits of a number to the left or right

instanceof
Used for type checking, to determine if an object is an instance of a particular class or interface

Operator precedence and associativity

运算符优先级和结合性

Precedence	Operators	Associativity	Description
1	`()`, `[]`, `.`, `::`	Left to Right	Parentheses, array access, member access, method reference
2	`++`, `--` (postfix)	Left to Right	Post-increment, post-decrement
3	`++`, `--`, `+`, `-`, `~`, `!` (prefix)	Right to Left	Pre-increment, pre-decrement, unary plus/minus, bitwise NOT, logical NOT
4	`(type)`, `new`	Right to Left	Type cast, object creation
5	`*`, `/`, `%`	Left to Right	Multiplication, division, modulus
6	`+`, `-`	Left to Right	Addition, subtraction
7	`<<`, `>>`, `>>>`	Left to Right	Bitwise shift operators
8	`<`, `<=`, `>`, `>=`, `instanceof`	Left to Right	Comparison, type check
9	`==`, `!=`	Left to Right	Equality, inequality
10	`&`	Left to Right	Bitwise AND
11	`^`	Left to Right	Bitwise XOR
12	`|`	Left to Right	Bitwise OR
13	`&&`	Left to Right	Logical AND
14	`||`	Left to Right	Logical OR
15	`? :`	Right to Left	Ternary conditional
16	`=`, `+=`, `-=`, `*=`, `/=`, `%=`, etc.	Right to Left	Assignment and compound assignment
17	`,`	Left to Right	Comma operator

Precedence	Operators	Associativity	Description
1	`()`, `[]`, `.`, `::`	Left to Right	Parentheses, array access, member access, method reference
2	`++`, `--` (postfix)	Left to Right	Post-increment, post-decrement
3	`++`, `--`, `+`, `-`, `~`, `!` (prefix)	Right to Left	Pre-increment, pre-decrement, unary plus/minus, bitwise NOT, logical NOT
4	`(type)`, `new`	Right to Left	Type cast, object creation
5	`*`, `/`, `%`	Left to Right	Multiplication, division, modulus
6	`+`, `-`	Left to Right	Addition, subtraction
7	`<<`, `>>`, `>>>`	Left to Right	Bitwise shift operators
8	`<`, `<=`, `>`, `>=`, `instanceof`	Left to Right	Comparison, type check
9	`==`, `!=`	Left to Right	Equality, inequality
10	`&`	Left to Right	Bitwise AND
11	`^`	Left to Right	Bitwise XOR
12	`|`	Left to Right	Bitwise OR
13	`&&`	Left to Right	Logical AND
14	`||`	Left to Right	Logical OR
15	`? :`	Right to Left	Ternary conditional
16	`=`, `+=`, `-=`, `*=`, `/=`, `%=`, etc.	Right to Left	Assignment and compound assignment
17	`,`	Left to Right	Comma operator

02

Arithmetic Operators

02

算术运算符

아주대학교 AJOU UNIVERSITY

Arithmetic Operators

- Arithmetic operators in Java are fundamental for performing mathematical calculations on numerical values.
- These operators work with both primitive and non-primitive data types, enabling basic operations like addition, subtraction, multiplication, division, and finding the remainder.

+	Addition	a + b	13
–	Subtraction	a - b	7
×	Multiplication	a * b	30
÷	Division	a / b	3
%	Remainder	a % b	1

```java
int a = 10;
int b = 3;
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a / b); // 3
System.out.println(a % b); // 1
```

算术运算符

- Java中的算术运算符对于对数值进行数学计算是基础。
- 这些运算符适用于原始和非原始数据类型，能够执行加法、减法、乘法、除法和求余等基本操作。

+	加法	a + b	13
–	减法	a - b	7
×	Multiplication	a * b	30
÷	Division	a / b	3
%	Remainder	a % b	1

```java
int a = 10;
int b = 3;
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a / b); // 3
System.out.println(a % b); // 1
```

Arithmetic Operators

算术运算符

Integer Division Result

When dividing two integers, Java performs integer division, which truncates any decimal part.

```
int a = 5, b = 2;
System.out.println(a / b);      // Result: 2
System.out.println((double)a / b); // Result: 2.5
```

Remainder with Negative

The sign of the result matches the sign of the dividend (first operand).

```
System.out.println(-7 % 3); // Result: -1
System.out.println(7 % -3);  // Result: 1
```

Data Type Overflow

Operations exceeding the maximum/minimum value of a data type result in overflow/underflow.

```
int x = Integer.MAX_VALUE;
System.out.println(x + 1); // Results in a negative number
```

String and + Operator

The + operator performs string concatenation if one operand is a String.

```
System.out.println("Result: " + 10 + 20);    // Result: 1020
System.out.println("Result: " + (10 + 20)); // Result: 30
```

整数除法结果

当除以两个整数时，Java 执行整数除法，这会截断任何小数部分。

```
int a = 5, b = 2; System.out.println(a / b); //
结果：2 System.out.println((double)a /
b); // 结果：2.5
```

负数的余数

结果的符号与被除数（第一个操作数）的符号相同。

```
System.out.println(-7 % 3); // 结果：-1
System.out.println(7 % -3); // 结果：1
```

数据类型溢出

超出数据类型的最大/最小值会导致溢出/下溢。

```
int x = Integer.MAX_VALUE; Syste
m.out.println(x + 1); // 结果为负数
```

字符串和 + 运算符

The + 运算符在其中一个操作数为 String 时执行字符串连接。

```
System.out.println("Result: " + 10 + 20); // Result: 1020
System.out.println("Result: " + (10 + 20)); // Result: 30
```

03

Assignment Operators

03

赋值运算符

아주대학교 AJOU UNIVERSITY

Assignment Operators

Used to store values in variables

Basic Assignment

The = operator assigns the value on its right to the variable on its left.

```
inta = 5;// a now holds the value 5
```

Compound Assignment

Combines an arithmetic operation with assignment.

Operator	Description	Example	Equivalent
+=	Add and assign	a += 3	a = a + 3
-=	Subtract and assign	a -= 2	a = a - 2
*=	Multiply and assign	a *= 4	a = a * 4

Examples

```
inta = 10;// Initial value: 10
a += 3; // a = a + 3 = 13
a -= 2; // a = a - 2 = 11
a *= 4; // a = a * 4 = 44
a /= 2; // a = a / 2 = 22
a %= 3; // a = a % 3 = 1
```

Key Points

- Compound assignment operators automatically handle implicit type casting
- They first perform the operation, then assign the result
- They are commonly used in loops and iterative calculations

赋值运算符

用于在变量中存储值

基本赋值

= 运算符将其右侧的值赋给左侧的变量。

```
inta = 5;// a现在持有值5
```

复合赋值

将算术运算与赋值结合。

操作符	描述	示例	等效
+=	加和赋值	a += 3	a = a + 3
-=	减去和 assign	a -= 2	a = a - 2
*=	乘以并赋值	a *= 4	a = a * 4

示例

```
inta = 10:// 初始值: 10
a += 3; // a = a + 3 = 13
a -= 2; // a = a - 2 = 11
a *= 4; // a = a * 4 = 44
a /= 2; // a = a / 2 = 22
a %= 3; // a = a % 3 = 1
```

关键点

- 复合赋值运算符自动处理隐式类型 casting
- 他们首先执行操作，然后分配结果
- 它们通常用于循环和迭代计算

04

Increment and Decrement Operators

04

增量与减量运算符

아주대학교 AJOU UNIVERSITY

Increment/Decrement

- Increment (++) and decrement (--) operators increase or decrease the value of a variable by one.
- They can be used before (prefix) or after (postfix) the variable.

Operator	Description	Example
++a → Prefix	Increments value, then returns it	int a = 5;System.out.println(++a); // 6 6
a++ → Postfix	Returns value, then increments it	int a = 5;System.out.println(a++); // 5 5
--a → Prefix	Decrements value, then returns it	int a = 5;System.out.println(--a); // 4
a-- → Postfix	Returns value, then decrements it	int a = 5;System.out.println(a--); // 5

增量/减量

- 增量 (++) 和减量 (--) 运算符增加或减少变量的值一个。
- 它们可以用于变量之前（前缀）或之后（后缀）。

操作员	描述	示例
++一个前缀	递增值，然后返回它	int a = 5;System.out.println(++a); // 6 6
a++ → 后缀	返回值，然后递增它	int a = 5;System.out.println(a++); // 5 5
--a 前缀	递减值，然后返回它	int a = 5;System.out.println(--a); // 4
a-- 后缀	返回值，然后递减它	int a = 5;System.out.println(a--); // 5

Increment/Decrement Cautions

⚠️ Difference Between Prefix (++a) and Postfix (a++)

Postfix (a++): Uses current value, then increments

```
int a = 5;
System.out.println(a++); // Outputs 5
System.out.println(a);   // Outputs 6
```

Prefix (++a): Increments first, then outputs

```
int a = 5;
System.out.println(++a); // Outputs 6
System.out.println(a);   // Outputs 6
```

🚫 Avoid Using in Complex Expressions

Using increment/decrement in complex expressions can lead to:

- Unpredictable results
- Difficult-to-read code
- Bugs that are hard to find

```
int i = 5;
int result = i++ + ++i; // Confusing!
// Better to avoid this pattern
```

Best Practice
Use increment/decrement operators in simple statements, not in complex expressions. When in doubt, use separate statements for clarity.

Increment/Decrement Cautions

⚠️ Prefix (++a) 和 Postfix (a++) 的区别

Postfix (a++): 使用当前值，然后自增

```
int a = 5;
System.out.println(a++); // 输出 5
System.out.println(a); // 输出 6
```

前缀 (++a): 先自增，再输出

```
int a = 5;
System.out.println(++a); // 输出 6
System.out.println(a); // 输出 6
```

🚫 避免在复杂表达式中使用

在复杂表达式中使用自增/自减可能导致：

- 不可预测的结果
- 难以阅读的代码
- 难以发现的错误

```
int i = 5;
int result = i++ + ++i; // 令人困惑！
// 避免这种模式更好
```

最佳实践
在简单语句中使用自增/自减运算符，而不是在复杂表达式中。如有疑问，请使用单独的语句以提高清晰度。

05

Comparison, Logical, Conditional

05

比较，逻辑，条件

아주대학교 AJOU UNIVERSITY

Comparison Operator

- Comparison operators evaluate relationships between two values, returning true or false.

= Equal to

`==`

Checks if two values are equal

```
System.out.println(a == b); // false
```

≠ Not equal to

`!=`

Checks if two values are not equal

```
System.out.println(a != b); // true
```

> Greater than

`>`

Checks if left value is greater

```
System.out.println(a > b); // false
```

< Less than

`<`

Checks if left value is smaller

```
System.out.println(a < b); // true
```

≥ Greater than or equal to

`>=`

Checks if left value is greater or equal

```
System.out.println(a >= b); // false
```

≤ Less than or equal to

`<=`

Checks if left value is smaller or equal

```
System.out.println(a <= b); // true
```

Commonly used in if

比较运算符

- 比较运算符用于评估两个值之间的关系，返回 true 或 false。

= 等于

`==`

检查两个值是否相等

```
System.out.println(a == b); // false
```

≠ 不等于

`!=`

检查两个值是否不相等

```
System.out.println(a != b); // true
```

> 大于

`>`

检查左值是否大于

```
System.out.println(a > b); // false
```

< 小于

`<`

检查左值是否更小

```
System.out.println(a < b); // true
```

≥ 大于或等于

`>=`

检查左值是否大于或等于

```
System.out.println(a >= b); // false
```

≤ 小于或等于

`<=`

检查左值是否小于或等于

```
System.out.println(a <= b); // true
```

常用于if

Comparison Operator Cautions

String Comparison

== checks **reference equality**

.equals() checks **value equality**

```
String str1 = "Hello";
String str2 = "Hello";
String str3 = new String("Hello");

// true (same reference in string pool)
System.out.println(str1 == str2);
// false (different objects in memory)
System.out.println(str1 == str3);
// true (correct value comparison)
System.out.println(str1.equals(str3));
```

Common Mistake:
Using **==** for string content comparison instead of **.equals()**

Floating-point Precision

Due to binary representation, floating-point numbers may have precision errors

```
double a = 0.1 + 0.2;

// false (due to floating-point representation inaccuracies)
```

Best Practice:
When comparing floating-point numbers, check if the absolute difference is less than a small tolerance (epsilon)

比较运算符注意事项

字符串比较

== checks **reference equality**

.equals() checks **value equality**

```
String str1 = "Hello";
String str2 = "Hello";
String str3 = new String("Hello");

// true (字符串池中相同的引用)
System.out.println(str1 == str2); // false (内存中不同的对象) System.out.println(str1 == str3); // true (正确的值比较)
System.out.println(str1.equals(str3));
```

常见错误：
使用 **==** 进行字符串内容比较，而不是 .equals()

浮点精度

由于二进制表示，浮点数可能存在精度误差

```
double a = 0.1 + 0.2;

// false (由于浮点数表示不准确)
```

最佳实践：
在比较浮点数时，检查绝对差是否小于一个小的容差（epsilon）

Logical Operator

- Logical operators are used to combine or modify boolean expressions, returning a boolean result.
- They are crucial for constructing complex conditions in if statements, while loops, and other control flow constructs.

&& Logical AND

Both operands must be true for the entire expressi on to be true.

true && true = true
true && false = false

|| Logical OR

At least one operand must be true for the entire ex pression to be true.

true || false = true
false || false = false

! Logical NOT

Inverts the boolean value of the operand.

! true = false
! false = true

逻辑运算符

- 逻辑运算符用于组合或修改布尔表达式，返回布尔结果。
- 它们对于在if语句、while循环和其他控制流结构中构建复杂条件至关重要。

&& 逻辑与

整个表达式为真，两个操作数都必须为真。

true && true = true
true && false = false

|| 逻辑或

整个表达式要为真，至少有一个操作数必须为真。

true || false = true
false || false = false

! 逻辑非

反转操作数的布尔值。

! true = false
! false = true

Logical Operator Cautions

⚠ Short-Circuit Evaluation

Logical operators **&&** and **||** exhibit short-circuit evaluation:

- **&&**: If left operand is **false**, right operand is **not evaluated**
- **||**: If left operand is **true**, right operand is **not evaluated**

This prevents NullPointerException and ArithmeticException

```
int a = 0;
if (a != 0 && (10 / a) > 1) {
   // Safe: (10 / a) not evaluated
}

int b = 5;
if (b == 5 || (10 / b) > 1) {
   // Safe: (10 / b) not evaluated if b == 5 is true
}
```

🔀 Avoid Confusion with Bitwise Operators

Logical operators vs. Bitwise operators:

Logical	Bitwise	Description
&&	&	Logical AND
||	|	Logical OR

*Bitwise operators evaluate **both** operands, which can cause unexpected errors*

```
boolean val1 = false;
boolean val2 = true;

System.out.println(val1 && val2); // false (logical AND)
System.out.println(val1 & val2);  // false (bitwise AND)

int x = 0;
if (x == 0 | (10 / x) > 1) {
   // This would cause an ArithmeticException
}
```

逻辑运算符注意事项

⚠ 短路求值

逻辑运算符 && 和 || 表现出短路求值：

- &&: 如果左操作数为 false，则不计算右操作数
- ||: 如果左操作数为 true，则不计算右操作数

这可以防止 NullPointerException 和 ArithmeticException

```
int a = 0;
if (a != 0 && (10 / a) > 1) {
   // Safe: (10 / a) not evaluated
}

int b = 5;
if (b == 5 || (10 / b) > 1) {
   // Safe: (10 / b) not evaluated if b == 5 is true
}
```

🔀 避免与位运算符混淆

逻辑运算符与位运算符：

逻辑	位运算	描述
&&	&	逻辑与
||	|	逻辑或

按位运算符会评估两个操作数，这可能导致意外的错误

```
boolean val1 = false;
boolean val2 = true;

System.out.println(val1 && val2); // false (逻辑与)
System.out.println(val1 & val2); // false (按位与)

int x = 0; if (x == 0 | (10 / x) > 1) { // 这将导
致一个 ArithmeticException }
```

Conditional (Ternary) Operator

What is the Conditional Operator?

The conditional operator provides a concise way to evaluate a condition and return one of two values based on whether the condition is true or false.

It is a shorthand for a simple **if-else** statement that assigns a value.

Format

```
condition ? value1 : value2
```

- **condition** - A boolean expression
- **value1** - Returned if condition is true
- **value2** - Returned if condition is false

Advantages

- More concise than if-else statements
- Can be used directly in expressions
- Readability issues with nested ternary operators

Example

```
int a = 10;
int b = 20;
int max = (a > b) ? a : b;
// max is 20
```

```
int max = (a > b) ?
  a :
  b;
```

vs

```
int max;
if (a > b) {
  max = a;
} else {
  max = b;
}
```

> ⚠ **Readability Cautions**
> Avoid nested ternary operators as they can become difficult to read quickly.
> Consider using if-else statements for complex conditions.

条件 (三元) 运算符

什么是条件运算符?

条件运算符提供了一种简洁的方式来评估一个条件，并根据条件是否为真或假返回两个值中的一个。条件为真或假。

它是简单 if-else 语句的简写，用于赋值。

格式

```
condition ? value1 : value2
```

- condition - 布尔表达式
- value1 - 如果 condition 为 true 则返回
- 值2 - 如果条件为假则返回

优点

- 比 if-else 语句更简洁
- 可以直接用于表达式
- 嵌套三元运算符的可读性问题

示例

```
int a = 10; int b = 20; int
max = (a > b) ? a : b; // max i
s 20
```

```
int max = (a > b) ?
  a :
  b;
```

vs

```
int max; if (a
> b) { max =
a; } else {
max = b; }
```

> ⚠ **可读性注意事项**
> 避免使用嵌套三元运算符，因为它们可能难以快速阅读。
> 对于复杂条件，请考虑使用 if-else 语句。

Quiz

- Operators are classified according to the number of operands they use. Which of the following is NOT a valid classification?
  - A. Unary operator
  - B. Binary operator
  - C. Ternary operator
  - D. Quaternary operator

Quiz

- 运算符根据其使用的操作数数量进行分类。以下哪项不是有效的分类?
  - A. 一元运算符
  - B. 二元运算符
  - C. 三元运算符
  - D. 四元运算符

Quiz

- When operators have the same precedence, which rule determines the order of execution?
  - A. Associativity
  - B. Operand type
  - C. Operator kind
  - D. Code writing order

Quiz

- 当运算符具有相同的优先级时，哪个规则决定了执行顺序?
  - A. 结合性
  - B. 操作数类型
  - C. 运算符种类
  - D. 代码书写顺序

Quiz

- In programming, when dividing two integers (e.g., 5 / 2), how is the result usually handled?
  - A. Rounded to the nearest whole number
  - B. Fractional part is discarded (truncated)
  - C. An error occurs
  - D. Automatically converted to a floating-point value

Quiz

- 在编程中，当除以两个整数（例如，5 / 2）时，结果通常如何处理?
  - A. 四舍五入为最接近的整数
  - B. 舍弃小数部分（截断）
  - C. 发生错误
  - D. 自动转换为浮点数

Quiz

- What does the operator represented by the % symbol calculate?
  - A. The quotient of division
  - B. The result of multiplication
  - C. The remainder of division
  - D. The square root

Quiz

- %符号代表的运算符计算什么?
  - A. 除法的商
  - B. 乘法的结果
  - C. 除法的余数
  - D. 平方根

Quiz

- When an operation is performed between different data types (e.g., integer and floating-point), what does the result's data type generally follow?
  - A. The wider data type
  - B. The narrower data type
  - C. The operator type
  - D. Compiler settings

Quiz

- 当在不同数据类型（例如整数和浮点数）之间执行操作时，结果的数据类型通常遵循什么？
  - A. 更宽的数据类型
  - B. 更窄的数据类型
  - C. 运算符类型
  - D. 编译器设置

Quiz

- What happens when you concatenate a string and a number using the + operator?
  - A. Arithmetic addition (number type)
  - B. An error occurs
  - C. The number is converted to a string and concatenated
  - D. The result is converted to a boolean value

Quiz

- 当您使用 + 运算符将字符串和数字连接在一起时会发生什么?
  - A. 算术加法（数字类型）
  - B. 发生错误
  - C. 数字被转换为字符串并连接
  - D. 结果被转换为布尔值

Quiz

- From the perspective of how a CPU performs division (repeated subtraction), what is the fundamental risk of dividing by zero?
  - A. Out of memory
  - B. Infinite loop
  - C. Data type mismatch
  - D. Operator precedence error

Quiz

- 从CPU执行除法（重复减法）的角度来看，除以零的基本风险是什么？
  - A. 内存不足
  - B. 无限循环
  - C. 数据类型不匹配
  - D. 运算符优先级错误

Quiz

- What is the essential requirement for something to be used as the left-hand side (L-value) of a simple assignment operator (=)?
  - A. A constant value
  - B. The result of an operation
  - C. A memory location that can store a value (variable)
  - D. A function call

Quiz

- 作为简单赋值运算符(=)的左侧（L-value）使用时，某物的基本要求是什么？
  - A. 一个常量值
  - B. 一个操作的结果
  - C. 一个可以存储值的内存位置（变量）
  - D. 一个函数调用

Quiz

- What is the main purpose of using the final keyword when declaring a variable?
  - A. To limit the variable name length
  - B. To prevent variable initialization
  - C. To make the variable immutable (constant)
  - D. To change the way memory is allocated for the variable

Quiz

- 使用 final 关键字声明变量时的主要目的是什么?
  - A. 限制变量名长度
  - B. 防止变量初始化
  - C. 使变量不可变（常量）
  - D. 改变变量内存分配的方式

Quiz

- What is the fundamental reason for commonly using a temporary variable (temp) when swapping the values of two variables A and B?
  - A. To speed up the operation
  - B. To prevent loss of A or B's value
  - C. To improve code readability
  - D. To prevent compiler errors

Quiz

- 为什么在交换两个变量 A 和 B 的值时通常使用临时变量（temp）？
  - A. 为了加快操作速度
  - B. 为了防止 A 或 B 的值丢失
  - C. 为了提高代码可读性
  - D. 为了防止编译器错误

Quiz

- What is the main difference between the prefix (++) and postfix (++) forms of the increment/decrement operators?
  - A. They apply to different variable types.
  - B. Their operator precedence is completely opposite.
  - C. The timing when the variable's value is used in the expression.
  - D. The postfix form can be used with constants.

Quiz

- 前缀（++）和后缀（++）形式的增减运算符的主要区别是什么？
  - A. 它们适用于不同的变量类型。
  - B. 它们的运算符优先级完全相反。
  - C. 变量的值在表达式中使用的时间。
  - D. 后缀形式可以与常量一起使用。

Quiz

- What is a major issue that can occur during type casting in programming?
  - A. A data type mismatch error always occurs.
  - B. Some of the original information may be lost (e.g., value truncation).
  - C. Operator precedence changes and causes unexpected results.
  - D. It cannot be applied to primitive types.

Quiz

- 编程中类型转换可能出现的重大问题是什么？
  - A. 总会发生数据类型不匹配错误。
  - B. 部分原始信息可能会丢失（例如，值截断）。
  - C. 运算符优先级发生变化并导致意外结果。
  - D. 它不能应用于基本类型。

Quiz

- What is the biggest difference between bitwise logical operators (&, |, ^) and regular logical operators (&&, ||)?
  - A. The result types are different.
  - B. Bitwise operators are only for integers, logical operators are only for floating-point numbers.
  - C. Bitwise operators work on each bit, while logical operators evaluate the overall boolean value.
  - D. Bitwise operators are unary, logical operators are binary.

Quiz

- 位运算逻辑符（&, | ,^）和常规逻辑符（&&, ||）之间最大的区别是什么？
  - A. 结果类型不同。
  - B. 位运算符仅用于整数，逻辑运算符仅用于浮点数。
  - C. 位运算符逐位操作，而逻辑运算符评估整体布尔值。
  - D. 位运算符是一元的，逻辑运算符是二元的。

Quiz

- For integer variables, using the left bit shift operator (<<) generally has the same effect as which arithmetic operation?
  - A. Addition
  - B. Subtraction
  - C. Multiplication
  - D. Division

Quiz

- 对于整数变量，使用左移位运算符(<<)通常与哪种算术运算具有相同的效果?
  - A. 加法
  - B. 减法
  - C. 乘法
  - D. 除法

Quiz

- When you compare two values using relational operators (e.g., >, ==, <=), what is the data type of the result?
  - A. Integer
  - B. Floating-point
  - C. String
  - D. Boolean

Quiz

- 当您使用关系运算符比较两个值时（例如，>, ==, <=), 结果的数据类型是什么？
  - A. 整数
  - B. 浮点数
  - C. 字符串
  - D. 布尔值

Quiz

- Why is it not recommended to compare floating-point numbers using relational operators '==' or '!=' for exact equality?
  - A. Floating-point numbers cannot be compared by size.
  - B. Due to internal floating-point representation errors.
  - C. You must always cast to integer before comparison.
  - D. Floating-point operators have very low precedence.

Quiz

- 为什么不推荐使用关系运算符 '==' 或 '!=' 来进行浮点数的精确相等比较?
  - A. 浮点数不能按大小进行比较。
  - B. 由于内部浮点数表示错误。
  - C. 比较前必须始终转换为整数。
  - D. 浮点数运算符的优先级非常低。

Quiz

- What is the most common result type of expressions used as operands for logical operators (&&, ||, !)?
  - A. Numeric type (integer or float)
  - B. String type
  - C. Boolean type
  - D. Array type

Quiz

- 逻辑运算符（&&，||，！）作为操作数的表达式最常见的结果类型是什么？
  - A. 数字类型（整数或浮点数）
  - B. 字符串类型
  - C. 布尔类型
  - D. 数组类型

Quiz

- In short-circuit evaluation, when using the logical AND operator (&&), when is the second operand not evaluated and the final result determined?
  - A. When the first operand is true
  - B. When the first operand is false
  - C. When both operands are true
  - D. When both operands are false

Quiz

- 简而言之，在使用逻辑与运算符（&&）时，当第二个操作数不被评估且最终结果已确定时是什么情况?
  - A. 当第一个操作数为真
  - B. 当第一个操作数为假
  - C. 当两个操作数都为真
  - D. 当两个操作数都为假

Quiz

- In short-circuit evaluation, when using the logical OR operator (||), when is the second operand not evaluated and the final result determined?
  - A. When the first operand is true
  - B. When the first operand is false
  - C. When both operands are true
  - D. When both operands are false

Quiz

- 简而言之，在使用逻辑 OR 运算符 (||) 时，当第二个操作数不被评估且最终结果已确定时是什么时候?
  - A. 当第一个操作数为真时
  - B. 当第一个操作数为假时
  - C. 当两个操作数都为真时
  - D. 当两个操作数都为假时

Quiz

- What is the core idea of the "survival" method for finding the maximum value among several numbers?
  - A. Store all numbers in an array and then sort.
  - B. Repeatedly compare pairs of numbers.
  - C. Keep track of the current maximum value and update it by comparing with each new input.
  - D. Add up all numbers and calculate the average.

Quiz

- “生存”方法在寻找多个数字中的最大值时的核心思想是什么?
  - A. 将所有数字存储在数组中然后排序。
  - B. 重复比较数字对。
  - C. 跟踪当前最大值并通过与新输入的比较来更新它。
  - D. 将所有数字相加并计算平均值。

03

Practice

03

练习

아주대학교 AJOU UNIVERSITY

Practice

- 求平均值
  - 用户输入两个整数
  - 输出平均值（精度留到小数点之下两位）

```
请输入两个整数:   10 20
AVG: 15.00
```

练习

- 求平均值
  - 用户输入两个整数
  - 输出平均值（精度保留到小数点后两位）

```
请输入两个整数: 10 20
AVG: 15.00
```

Practice

- 计算时分秒
  - 用户输入整数的秒（Second）
  - 输出时分秒

```
请输入秒:   4000
4000秒是 1小时6分钟40秒
```

练习

- 计算时分秒
  - 用户输入整数的秒(Second)
  - 输出时分秒

```
请输入秒: 4000
4000秒是1小时6分钟40秒
```

Practice

- 两个变量的交换问题
  - 用户输入两个整数
  - 用变量x0 和 x1 存储用户输入的整数
  - 进行交换后输出

```
请输入 x0: 3
请输入 x1: 4
交换结果 x0: 4, x1: 3
```

练习

- 两个变量的交换问题
  - 用户输入两个整数

    使用变量x0和x1存储用户输入的整数
  - 进行交换后输出

```
请输入 x0: 3
请输入 x1: 4
交换结果 x0: 4, x1: 3
```

Practice

- 求三个整数的的总和
  - 用户输入三个整数
  - 求出三个数的总和并输出
  - 代码里只允许使用两个变量

练习

- 求三个整数的总和
  - 用户输入三个整数
  求出三个数的总和并输出
  - 代码里只允许使用两个变量

Practice

- 判断合格与否
  - 用户输入一个成绩
  - 若80分以上为合格，否则不合格
  - 不可以使用条件语句

请输入成绩： 88
结果： 合格

练习

- 判断合格与否
  - 用户输入一个成绩
  如果80分以上为合格，否则不合格
  - 不可以使用条件语句

请输入成绩: 88 结果: 合格

Practice

- 求出最大值
- 用户输入三个整数
- 用两种方法求出最大值
- 不可以使用条件语句

```
请输入三个数字:   10 20 30
MAX:  30
```

练习

- 求出最大值
- 用户输入三个整数

使用两种方法求出最大值

- 不可以使用条件语句

```
请输入三个数字: 10 20 30
MAX:30
```

Q & A

问答

THANK YOU

谢谢